

Séance 1 – Introduction au cours

Programmation – CPES DAC

Laura Manuela Castañeda Medina

manuela.castaneda@psl.eu

Septembre 2026

C'est quoi un ordinateur ?

Définition (inspirée de Gérard Swinnen) :

À strictement parler, un ordinateur est une machine qui effectue des opérations simples sur des séquences de signaux électriques. Ces signaux ne peuvent prendre que deux états (par exemple : tension faible ou tension élevée). Ils obéissent à une logique de type *tout ou rien* et peuvent donc être représentés comme des suites de nombres binaires (0 ou 1).

- Ainsi, toute donnée ou instruction que l'ordinateur doit traiter doit être convertie en **format binaire**, que ce soit du texte, une image, un son, un programme, etc. l'ordinateur ne comprend que ça.

📖 Réf. : Gérard Swinnen, *Apprendre à programmer avec Python 3*

C'est quoi un langage de programmation ?

- L'ordinateur comprend uniquement le **langage machine** : des suites de 0 et de 1, traitées en groupes (8, 16, 32, 64 bits).
- Pour communiquer avec la machine, on utilise un **langage de programmation**, composé de mots-clés et de règles bien définies.
- Ces langages sont traduits en langage binaire par un **interpréteur** ou un **compilateur**.

📖 *Réf. : Gérard Swinnen, Apprendre à programmer avec Python 3*

Compilation vs Interprétation

Langage interprété (ex : Python)

- Le code est *traduit en langage machine au fur et à mesure de son exécution*.
- Pas besoin d'étape de compilation : plus simple pour tester et corriger rapidement.
- Très **portable** : le même script peut fonctionner sur Windows, Linux ou Mac sans modification.

Langage compilé (ex : C)

- Le code est *entièrement converti en langage machine avant exécution*.
- Nécessite un **compilateur**, à relancer à chaque modification du programme.
- Plus **rapide** à l'exécution, mais moins flexible et moins portable.

 Réf. : Vincent Le Goff, Apprenez à programmer en Python, OpenClassrooms

Langages haut vs bas niveau

- **Bas niveau** : très proche de la machine (ex : assembleur). Peu lisible, difficile à écrire, mais très rapide à exécuter.
- **Haut niveau** : proche du langage humain (ex : Python, Pascal, Lisp...). Plus facile à lire, à écrire et à maintenir.
- Chaque instruction haut niveau est traduite en plusieurs instructions machines.

 *Réf. : Gérard Swinnen, Apprendre à programmer avec Python 3*

Pourquoi apprendre à programmer en 2026, alors qu'une IA peut écrire en trente secondes un programme qui nous prendrait une heure ?

L'IA est déjà présente

Dans l'enquête Stack Overflow 2025 :

- 84 % des répondants utilisent ou envisagent d'utiliser des outils d'IA pour programmer ;
- 51 % des développeurs professionnels déclarent les utiliser quotidiennement ;
- parmi les utilisateurs d'agents, environ 70 % estiment qu'ils réduisent le temps consacré à certaines tâches.

 *Réf. : Stack Overflow Developer Survey 2025*

Mais utiliser n'est pas faire confiance

Dans la même enquête :

- 46 % des développeurs disent se méfier de l'exactitude des réponses, contre 33 % qui leur font confiance ;
- 66 % citent comme principale difficulté les solutions « presque correctes » ;
- 45 % trouvent que déboguer du code généré par l'IA peut prendre davantage de temps.

 *Réf. : Stack Overflow Developer Survey 2025*

Ce que signifie « apprendre à coder » aujourd'hui

Avant, l'accent était surtout mis sur....	Aujourd'hui, il faut aussi savoir...
Mémoriser la syntaxe	Lire et modifier du code généré
Écrire chaque ligne soi-même	Décrire précisément le résultat attendu
Trouver une solution qui fonctionne	Construire des tests qui tentent de la faire échouer
Corriger ses propres erreurs	Détecter les erreurs plausibles d'une IA
Réaliser un programme	Expliquer ses choix et ses limites
Connaître un langage	Combiner programmation, métier et outils d'IA

Calendrier du cours

Séance	Date	Thèmes
1	23/09	Variables, <code>print()</code> et listes
2	07/10	Conditions, boucles et tests
3	03/11	Fichiers, fonctions et dictionnaires
4	18/11	Modules et organisation d'un projet Python
5	02/12	Présentations des projets
6	16/12	Présentations des projets
7	13/01	Examen final

Calendrier prévisionnel

Projet du semestre

À la fin du semestre, votre projet devra être un **programme Python complet et fonctionnel**.

Élément	Ce que j'attends
Objectif	Un objectif clair et un programme utilisable
Données	Listes et/ou dictionnaires pour organiser les données
Logique	Conditions et boucles utilisées de manière pertinente
Fonctions	Code décomposé et organisé en fonctions
Fichiers & modules	Lecture/sauvegarde de données et plusieurs fichiers <code>.py</code>
Tests	Tests permettant de vérifier le fonctionnement du programme

Comment le projet sera-t-il évalué ?

Fonctionnalité (50 % du projet)	Présentation (50 % du projet)
Le programme fonctionne	Objectif du projet clairement expliqué
Les notions du cours sont utilisées	Démonstration du programme
Le code est organisé et lisible	Explication des choix réalisés
Les cas principaux sont testés	Capacité à expliquer le code
Le projet respecte les consignes	Limites / améliorations possibles

Le projet évoluera avec le cours

Séance	Vous pourrez ajouter...
1	Données du projet avec variables et listes + affichage
2	Interactions, choix et répétitions avec conditions et boucles
3	Fonctions , dictionnaires et sauvegarde/lecture de fichiers
4	Organisation du code en plusieurs fichiers avec des modules
5	Finalisation, tests, corrections et préparation de la présentation



Évaluation

- 40 % — Examen final
- 60 % — Projet du semestre
 - 30 % — Fonctionnalité du projet
 - 30 % — Présentation du projet